

CSE3532 — Tools and Technologies for Internet Programming

Section A — Short Questions (2 Marks) — Quick Answers

ReactJS (Q1–Q12)

Q1. What is ReactJS and why is it used?

JavaScript library (Meta) for building UIs/SPAs. Used for component reusability, Virtual DOM speed, large ecosystem.

Q2. What is the Virtual DOM and how does it work?

Lightweight in-memory copy of real DOM. React diffs new vs old tree (reconciliation) and updates only changed parts.

Q3. What is JSX? Why is it used?

HTML-like syntax extension for JS. Easier to write UI; compiles to `React.createElement()`.

Q4. What is a Component? Types?

Reusable UI block returning JSX. Types: Functional (Hooks) and Class (`this.state`, lifecycle methods).

Q5. Difference between Props and State?

Props: parent→child, read-only. State: internal, mutable via `setState/useState`.

Q6. Why are props immutable?

Maintains unidirectional data flow; prevents child from altering parent data.

Q7. What is useState? How does it work?

Hook adding state to function components: `const [v,setV]=useState(init)`; `setV` updates & re-renders.

Q8. What is useEffect? Dependency array?

Hook for side effects (API calls etc). `[]` = run once; `[x]` = run on x change; `none` = every render.

Q9. What is useRef? Use cases?

Mutable ref object persisting without re-render. Used for DOM access, storing previous values, timers.

Q10. What is useContext? Why Context API?

Consumes Context data directly. Shares global data (theme/auth) without prop drilling.

Q11. What is React Router? Why used?

Library for client-side routing/navigation in SPAs without full page reload.

Q12. Difference: Link vs NavLink?

Link: basic navigation. NavLink: adds auto 'active' class/style for current route.

Firebase Authentication (Q13–Q20)

Q13. What is Firebase? Why is Auth important?

Google BaaS (Auth, Firestore, Hosting). Auth verifies identity & secures access.

Q14. How does Email & Password Auth work?

User signs up with email/password; Firebase creates account & verifies on login, returns user object.

Q15. Purpose of onAuthStateChanged?

Listener that triggers automatically on login/logout state change to keep UI in sync.

Q16. What is Google Sign-in in Firebase?

OAuth social login using `GoogleAuthProvider` + `signInWithPopup()`/`signInWithRedirect()`.

Q17. What is GitHub authentication?

OAuth login using `GithubAuthProvider` for developer-focused apps.

Q18. What is user persistence?

Determines if login persists across sessions: LOCAL, SESSION, or NONE.

Q19. What does `signInWithEmailAndPassword()` do?

Authenticates existing user by verifying email/password, returns credential.

Q20. What does `signOut()` do?

Logs out current user, clears session/auth state.

NodeJS & Express (Q21–Q31)

Q21. NodeJS vs browser JavaScript?

Node = JS runtime (V8) for server-side; has fs/OS access, no DOM (unlike browser JS).

Q22. Why single-threaded but handles many requests?

Uses event loop + non-blocking async I/O; heavy tasks offloaded to libuv thread pool.

Q23. What is ExpressJS? Why used?

Minimal Node web framework simplifying routing & middleware vs raw http module.

Q24. What is Middleware? Examples?

Function with (req,res,next) run during request cycle. E.g. `express.json()`, `cors()`, `morgan()`.

Q25. What is Routing in Express?

Defines app response to specific URL + HTTP method, e.g. `app.get('/students', handler)`.

Q26. What is CORS and why required?

Browser security mechanism controlling cross-origin access; middleware permits it explicitly.

Q27. What is nodemon and why used?

Dev tool auto-restarting server on code changes.

Q28. What is dotenv? Why env vars matter?

Loads .env variables into process.env; keeps secrets out of code, enables per-environment config.

Q29. What is a RESTful API?

API following REST principles: stateless, HTTP methods, resource URLs, usually JSON.

Q30. GET vs POST?

GET: retrieves data via URL/query, no side effects. POST: sends data in body, changes server state.

Q31. Purpose of `app.use()`?

Mounts middleware/routers to app, applied to all or specific routes.

REST API, Request/Response, Headers (Q32–Q40)

Q32. What is an API? What does REST stand for?

API = rules for software communication. REST = Representational State Transfer.

Q33. HTTP Request & its components?

Client-to-server message; components: Method, URL, Headers, Body (optional).

Q34. HTTP Response & its components?

Server's reply; components: Status Code, Headers, Body.

Q35. What are HTTP headers?

Key-value metadata pairs sent with request/response, e.g. Content-Type, Authorization.

Q36. What is a request body?

Actual data/payload sent by client, used with POST/PUT.

Q37. Query parameter vs path parameter?

Query: ?key=value (filtering). Path: /students/:id (identifies specific resource).

Q38. Status codes 200, 400, 401, 404, 500?

200 OK-success; 400 Bad Request-invalid input; 401 Unauthorized; 404 Not Found; 500 Server Error.

Q39. What is Content-Type header?

Specifies media/MIME type of body data (e.g. application/json), tells receiver how to parse it.

Q40. What is Authorization header?

Sends credentials (e.g. Bearer token) to authenticate/authorize access.

JWT – JSON Web Token (Q41–Q46)

Q41. What is JWT and why used?

Compact signed JSON token for secure info transfer; used for stateless auth/authorization.

Q42. Three parts of a JWT?

Header (algorithm/type) . Payload (claims) . Signature (integrity), joined by dots.

Q43. How does JWT auth work?

Login→server verifies & issues JWT→client stores & sends via Authorization header→server verifies signature.

Q44. Why is JWT stateless?

No server-side session storage; user info encoded in the token itself.

Q45. What is token expiration? Why important?

exp claim sets validity period; limits damage if token is stolen, forces re-auth.

Q46. Access token vs refresh token?

Access: short-lived, used per request. Refresh: long-lived, used to get new access token.

MongoDB & Database Concepts (Q47–Q50)

Q47. What is MongoDB? Why NoSQL?

Document-oriented DB storing BSON/JSON docs; NoSQL = no fixed relational schema.

Q48. Document vs Collection in MongoDB?

Document: single JSON-like record. Collection: group of documents (like a table).

Q49. SQL vs NoSQL differences?

SQL: fixed schema, tables, vertical scaling (MySQL). NoSQL: flexible schema, documents, horizontal scaling (MongoDB).

Q50. When choose SQL vs NoSQL?

SQL: structured data, complex relations, strong consistency (banking). NoSQL: unstructured, frequent schema change, high scalability.